GraphQL and API Federation

@FEUP-ASSO-2025-26C02-T3

01

GraphQL

What is GraphQL?

What is GraphQL?

A query language for APIs and a runtime for fulfilling those queries with your existing data.

Single Endpoint

All queries via one POST to /graphql - no more versioned URLs.

Type System (SDL)

Schema Definition Language defines every available field precisely.

Precision Fetching

No over-fetching, no-under-fetching: get exactly what you need.

Core Concept: REST is a **vending machine** (fixed items), **GraphQL** is a buffet (you pick exactly what goes on your plate).

The Vending Machine vs The Buffet

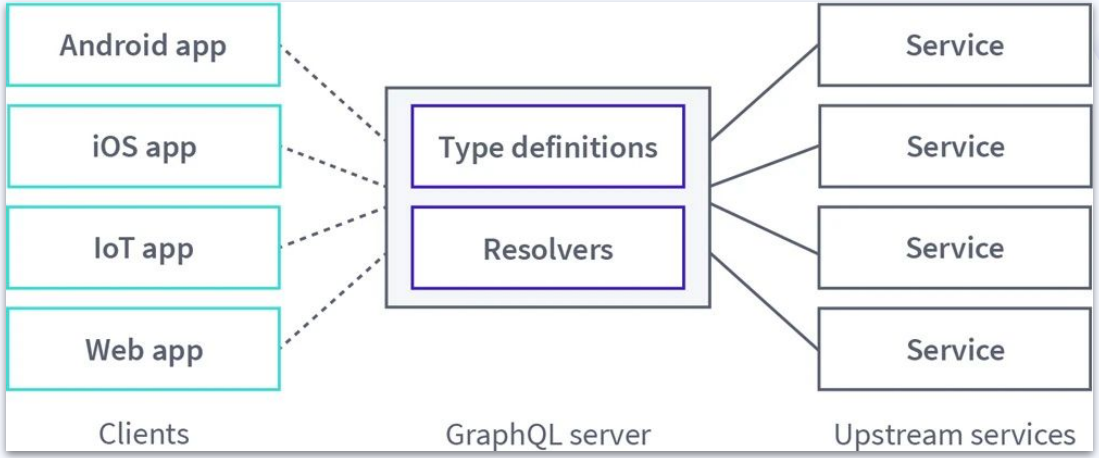
Feature	REST (Vending Machine)	GraphQL (The Buffet)
Endpoints	Multiple (/users, /products, /orders)	Single (/graphql)
Data Control	Server defines the response	Client defines the query
Relationships	Multiple requests (under-fetching)	Single nested query
Payload	Fixed structure (over-fetching)	Only requested fields returned

02

GraphQL Adoption Patterns

Monolith vs. GraphQL Federation

The Monolith Pattern



Best for: Startups and small-to-medium teams.

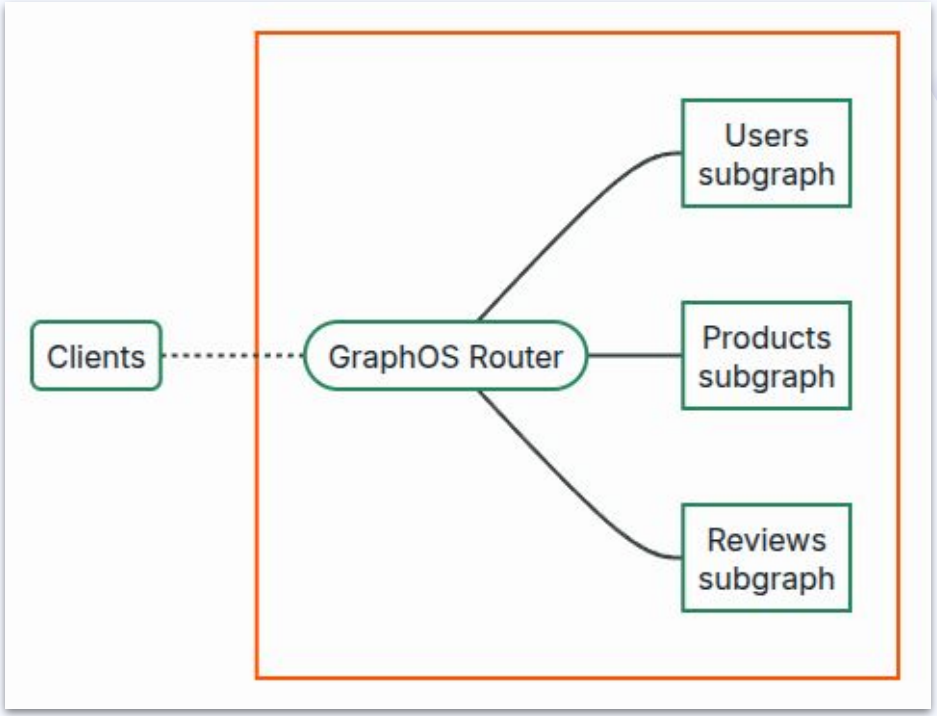
Advantages

- **Simplicity**: one codebase to manage;
- **Performance**: no internal latency;
- **Unified Graph**: all data relationships are easily defined in one place.

The "Wall": as the organization grows, Monolith becomes a bottleneck (constant merge conflicts).

API Federation

The Supergraph: a router sits in front of multiple Subgraphs (Microservices).



Advantages

- Distributed Ownership;
- Declarative Composition;
- **The Benefit**: it combines the autonomy of microservices with the usability of a monolith.

03

When to use GraphQL?

When to use GraphQL?

Complex Data

When UI needs deeply nested data in one request.

Mobile-First

When it needs to save bandwidth by fetching only the necessary fields.

Frontend Speed

Frontend teams can iterate on UI changes without waiting for backend.

04

When NOT to use GraphQL?

When NOT to use GraphQL?

Simple CRUD APIs

Over-engineering for basic data retrieval. Stick to REST for simple structures.

Heavy Caching

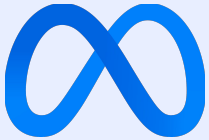
Single POST endpoints make standard HTTP caching significantly harder to implement.

05

Conclusion

Conclusion: Global Adoptions

The Pioneers



The Federation Leaders

The Scale-ups

GraphQL and Federation bridge a critical gap: allowing backend microservice autonomy while delivering a unified frontend experience.

Questions?

Thank you for your attention!